

Informe Práctica 2

DLX

Alex García Alegre
Sergio Hernández García

Índice

Ejercicio 1: Cifrado XOR.....	3
Cálculo de longitudes.....	3
Proceso de Cifrado/Descifrado.....	3
Gestión de registros.....	3
Gestión de Salida.....	3
Ejercicio 2: Optimización del Pipeline y Planificación.....	4
Como se ha ordenado y por qué.....	4
Estadísticas obtenidas.....	4
Comparación entre las estadísticas obtenidas.....	5
Ciclos obtenidos en la versión ordenada.....	6

Ejercicio 1: Cifrado XOR

Cálculo de longitudes

Se ha llevado a cabo un cálculo de longitudes, por medio de dos bucles, uno para el mensaje y otro para la clave.

Dichos bucles utilizan la instrucción LB (Load Byte) para traer un carácter de la memoria al registro y compararlo con el carácter nulo, mientras que se va aumentando el índice r1. Si se detecta el carácter nulo, por medio de la instrucción BEQZ (Branch Equal Zero) se finaliza el bucle, ya que hemos llegado al final de la cadena y se toma el valor de r1 como la longitud de la cadena.

Proceso de Cifrado/Descifrado

Hemos utilizado un esquema de cifrado de flujo, donde cada carácter del mensaje se combina con un carácter de la clave por medio de la instrucción XOR.

Para el caso de querer gestionar claves más cortas que el mensaje, hemos implementado un mecanismo de reinicio del índice, en el que se va restando el índice actual de la clave con la longitud total, de manera que si el resultado es igual a 0, se va a resetear el índice de la clave a 0 para volver a empezar el ciclo de la misma.

Al final de los procesos de cifrado y descifrado hemos añadido el carácter nulo a la cadena.

Gestión de registros

Vamos a tener varios registros de propósito general:

- r16: Índice global del mensaje, que funciona de puntero de desplazamiento.
- r17: Índice circular de la clave.
- r18: registro que nos permite llevar un control de frontera para la clave, donde nos permite evitar el desbordamiento.
- r19: registro que funciona de contador de iteraciones, basado en la longitud del mensaje.

También tendremos registros que nos permite llevar a cabo una impresión por pantalla:

- r14: registro que actúa como dirección para los descriptores de formato en las diferentes llamadas al sistema de nuestro programa.
- r30: registro de paso de parámetros para las funciones de impresión por pantalla.

Gestión de Salida

Se ha hecho uso de funciones auxiliares, las cuales han sido proporcionadas en el propio documento del enunciado de los ejercicios, y de la instrucción TRAP 5, para poder imprimir resultados intermedios en formato ASCII y hexadecimal, facilitando el seguimiento del algoritmo.

Ejercicio 2: Optimización del Pipeline y Planificación

Como se ha ordenado y por qué

Basándonos en el Instruction Scheduling, que nos permite ocultar la latencia de la unidad de división, ya que dicha configuración que se nos ha propuesto, la división ocupa 19 ciclos, y por medio de dicha optimización, se nos permite llevar a cabo otras operaciones mientras se resuelve la división.

En la versión sin ordenar, las divisiones se iban a lanzar de manera secuencial, provocando paradas estructurales grandes. Esto se arregla en la versión ordenada, ya que se han insertado instrucciones de carga (LF), multiplicaciones (MULTF), sumas (ADDF) y comparaciones de seguridad (EQF) mientras se ejecutan divisiones.

Cómo llevamos a cabo la separación del inicio de cada división, mínimo en 19 ciclos, con otras instrucciones útiles, evitamos que el pipeline se detenga por la ocupación de la unidad funcional de división.

Finalmente, se ha sustituido el salto condicional final por una estructura de flujo lineal hacia el TRAP 0, reduciendo así las paradas por saltos.

Estadísticas obtenidas

```
Total:
155 Cycle(s) executed.
ID executed by 38 Instruction(s).
2 Instruction(s) currently in Pipeline.

Hardware configuration:
Memory size: 32768 Bytes
faddEX-Stages: 1, required Cycles: 2
fmulEX-Stages: 1, required Cycles: 5
fddivEX-Stages: 1, required Cycles: 19
Forwarding enabled.

Stalls:
RAW stalls: 39 (25.16% of all Cycles), thereof:
  LD stalls: 1 (2.56% of RAW stalls)
  Branch/Jump stalls: 0 (0.00% of RAW stalls)
  Floating point stalls: 38 (97.44% of RAW stalls)
WAW stalls: 0 (0.00% of all Cycles)
Structural stalls: 74 (47.74% of all Cycles)
Control stalls: 1 (0.64% of all Cycles)
Trap stalls: 2 (1.29% of all Cycles)
Total: 116 Stall(s) (74.84% of all Cycles)

Conditional Branches):
Total: 5 (13.16% of all Instructions), thereof:
  taken: 0 (0.00% of all cond. Branches)
  not taken: 5 (100.00% of all cond. Branches)

Load-/Store-Instructions:
Total: 8 (21.05% of all Instructions), thereof:
  Loads: 7 (87.50% of Load-/Store-Instructions)
  Stores: 1 (12.50% of Load-/Store-Instructions)

Floating point stage instructions:
Total: 18 (47.37% of all Instructions), thereof:
  Additions: 8 (44.44% of Floating point stage inst.)
  Multiplications: 5 (27.78% of Floating point stage inst.)
  Divisions: 5 (27.78% of Floating point stage inst.)

Traps:
Traps: 1 (2.63% of all Instructions)
```

```
Total:
113 Cycle(s) executed.
ID executed by 37 Instruction(s).
2 Instruction(s) currently in Pipeline.

Hardware configuration:
Memory size: 32768 Bytes
faddEX-Stages: 1, required Cycles: 2
fmulEX-Stages: 1, required Cycles: 5
fddivEX-Stages: 1, required Cycles: 19
Forwarding enabled.

Stalls:
RAW stalls: 22 (19.47% of all Cycles), thereof:
  LD stalls: 1 (4.54% of RAW stalls)
  Branch/Jump stalls: 0 (0.00% of RAW stalls)
  Floating point stalls: 21 (95.45% of RAW stalls)
WAW stalls: 0 (0.00% of all Cycles)
Structural stalls: 49 (43.36% of all Cycles)
Control stalls: 0 (0.00% of all Cycles)
Trap stalls: 7 (6.19% of all Cycles)
Total: 78 Stall(s) (69.03% of all Cycles)

Conditional Branches):
Total: 5 (13.51% of all Instructions), thereof:
  taken: 0 (0.00% of all cond. Branches)
  not taken: 5 (100.00% of all cond. Branches)

Load-/Store-Instructions:
Total: 8 (21.62% of all Instructions), thereof:
  Loads: 7 (87.50% of Load-/Store-Instructions)
  Stores: 1 (12.50% of Load-/Store-Instructions)

Floating point stage instructions:
Total: 18 (48.65% of all Instructions), thereof:
  Additions: 8 (44.44% of Floating point stage inst.)
  Multiplications: 5 (27.78% of Floating point stage inst.)
  Divisions: 5 (27.78% of Floating point stage inst.)

Traps:
Traps: 1 (2.70% of all Instructions)
```

Comparación entre las estadísticas obtenidas

	SIN ORDENAR	ORDENADA
CYCLES	155	113
INSTRUCTION	38	37
LD STALLS	1	1
FLOATING POINT STALLS	38	21
STRUCTURAL STALLS	74	49
LOADS	7	7
STORES	1	1
CONDICIONAL BRANCH TAKEN	0	0
CONDICIONAL BRANCH NOT TAKEN	5	5
ADDITIONS	8	8
MULTIPLICATIONS	5	5
DIVISIONS	5	5
RESULTADO	12045.0	12045.0

Por otro lado, hemos calculado el número de ciclos por instrucción (CPI), para ambos casos.

$CPI_{sinOrdenar} = 155 \text{ ciclos} / 38 \text{ instrucciones} = 4,08$

$CPI_{Ordenado} = 113 \text{ ciclos} / 37 \text{ instrucciones} = 3,05$

Como podemos ver, gracias a la ordenación de operaciones, conseguimos reducir el coste por instrucción en casi un 25%. Por medio de dicha comparación podemos ver que cada ciclo de reloj es más productivo al reducir el tiempo que el procesador pasa en estado de stall.

Ciclos obtenidos en la versión ordenada

El resultado obtenido en la versión ordenada es de 113 ciclos, este es el límite óptimo para dicho código, ya que las restricciones físicas del hardware configurado así nos lo dicen. Podemos justificar este número matemáticamente de la siguiente manera:

El programa debe ejecutar obligatoriamente 5 divisiones. Dado que la latencia es de 19 ciclos por operación y solo disponemos de una unidad funcional no segmentada, el procesador requiere un mínimo absoluto de 95 ciclos (5×19) solo para completar las divisiones.

Gracias a la reordenación, prácticamente la totalidad de las 7 cargas, 8 sumas, 5 multiplicaciones y comparaciones se ejecutan "ocultas" dentro de esos 95 ciclos, aprovechando los huecos de latencia del hardware.

Finalmente, la diferencia entre los 95 ciclos de división y los 113 totales se debe a el llenado inicial del pipeline (etapas IF e ID de las primeras instrucciones), las latencias de las operaciones finales que presentan dependencias de datos (RAW) insalvables, como el gran producto final que une los bloques de la ecuación y que debe esperar a que los resultados previos estén disponibles en el banco de registros, y el vaciado del pipeline (etapas MEM y WB) necesario para que la última instrucción se complete antes de alcanzar el TRAP 0.

En conclusión, mediante la planificación estática y el aprovechamiento del adelantamiento de datos (Forwarding), se ha logrado una ejecución óptima donde el procesador realiza trabajo útil en casi cada ciclo de reloj disponible.